

Deep Learning para Business Analytics

Día 07 · Tema 2 — Atención, puente al Transformer

Por qué la recurrencia ya no es suficiente.

Eduardo C. Garrido-Merchán

Profesor Propio Adjunto, Métodos Cuantitativos, ICADE

ecgarrido@comillas.edu



Madrid, Día 07



ejemplos del día

Lo que vamos a cubrir hoy.

1. Las RNN y su límite
2. La idea de la atención
3. RNN vs atención, comparativa
4. Hoja de ruta

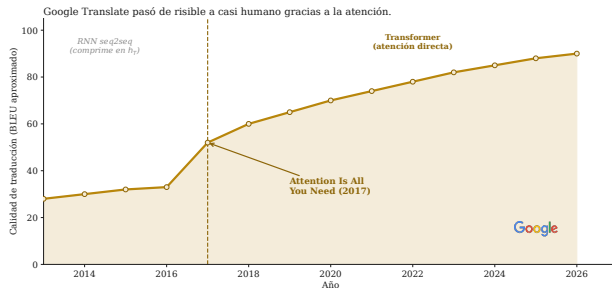
Al final del día: entiendes la 🧠 atención sobre una frase concreta y calculas sus pesos a mano. El Día 09 te enseña el ⚡ Transformer completo.

Parte 1 / 4

Las RNN y su límite



🗨️🚀 Google Translate pasó de risible a casi humano en 2017 gracias a la atención.



Antes de la atención. La RNN comprimía una frase de 50 palabras en un solo vector h_T . Imagina resumir *El Quijote* en un post-it.

El punto de inflexión. En 2017, “Attention Is All You Need” permitió al decoder mirar directamente cada posición del encoder. La calidad de traducción se disparó.

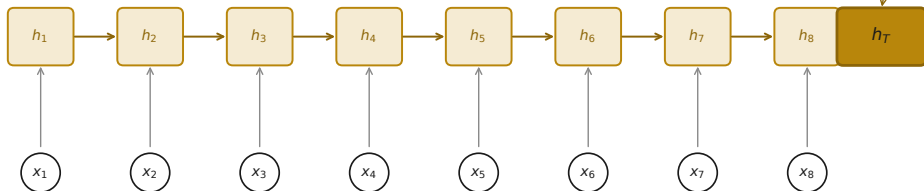
Hoy. Google Translate, DeepL y todos los traductores modernos usan atención. Sin ella, los modelos de lenguaje actuales no existirían.

IDEA CLAVE. La atención resolvió el cuello de botella que impedía traducir frases largas.

! Cuello de botella: toda la historia cabe en un solo vector.

Cuello de botella de la RNN: 100 pasos comprimidos en un solo vector.

toda la historia
comprimida aquí



Por qué duele el bottleneck.

Información comprimida.  Con $h_T \in \mathbb{R}^{64}$ hay que resumir 100 pasos de historia en 64 números. Para tareas con dependencias largas (traducción de un párrafo, predicción de demanda a 12 meses), esos 64 números no llegan.

Paralelismo limitado.  Para calcular h_{100} hay que pasar por $h_1, h_2, \dots, h_{99}$ en orden. La GPU se queda esperando.

Olvido en la retropropagación.  El gradiente de $\partial \mathcal{L} / \partial w$ se calcula multiplicando matrices a lo largo de toda la cadena temporal; para secuencias largas se desvanece o explota.

IDEA CLAVE. La atención resuelve los tres problemas al mismo tiempo.

Parte 2 / 4

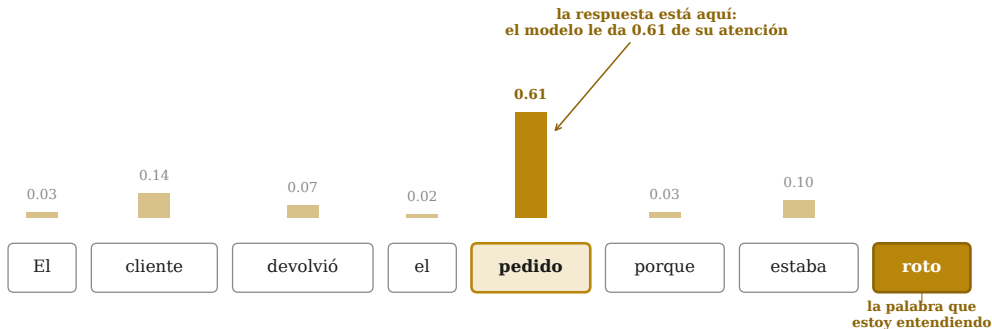
La idea de la atención



☞ Para entender “roto” hay que mirar a otra palabra de la frase.

“El cliente devolvió el pedido porque estaba roto.”

Para entender “roto” hay que saber qué estaba roto. ¿A qué palabra de la frase hay que mirar?



La altura de cada barra es cuánta atención recibe esa palabra. Las ocho alturas suman 1.

IDEA CLAVE. El modelo reparte 1 punto de atención entre las ocho palabras. “pedido” se lleva 0.61 y “cliente” 0.14; los artículos, casi nada. Eso es toda la idea: leer una palabra mirando a las que la explican.

Por qué gana “pedido”: cada palabra ofrece algo y “roto” busca algo.

QUERY · la palabra que estoy entendiendo y lo que necesita saber

roto busco algo de la frase que pueda estar roto

KEY · lo que cada palabra ofrece

atención

VALUE · lo que aporta a “roto”

pedido

“una cosa, un objeto que se puede romper”



0.61

el pedido

cliente

“una persona, no se rompe”



0.14

el cliente

estaba

“un verbo de estado en pasado”



0.10

ocurrió antes

devolvió

“la acción principal de la frase”



0.07

la devolución

El, el, porque

“artículos y conectores, sin contenido”



0.08

casi nada

“roto” pasa a significar $0.61 \cdot (\text{el pedido}) + 0.14 \cdot (\text{el cliente}) + \dots = \text{“roto, referido al pedido”}$

IDEA CLAVE. Esas tres cosas tienen nombre: lo que la palabra busca es su **query**, lo que cada palabra ofrece es su **key**, y lo que aporta si gana es su **value**. El peso α sale de comparar la query con cada key.

Cambia la frase y “banco” mira a otro sitio, así que significa otra cosa.

La misma palabra en dos frases: “banco” mira a sitios distintos y acaba significando cosas distintas.



Ninguna regla dice qué significa “banco”: lo decide a qué palabras de su propia frase presta atención.

La misma palabra entra igual y sale distinta según la frase.

Antes de la atención



↓ la atención mezcla el contexto de la frase

Después de la atención
en "El banco subió los tipos de interés."



en "Nos sentamos en el banco del parque."



Frase 1. "El banco subió los tipos de interés." La query de "banco" encaja con "interés" (0.56) y "tipos" (0.26).

Frase 2. "Nos sentamos en el banco del parque." La misma query encaja ahora con "parque" (0.61) y "sentamos" (0.27).

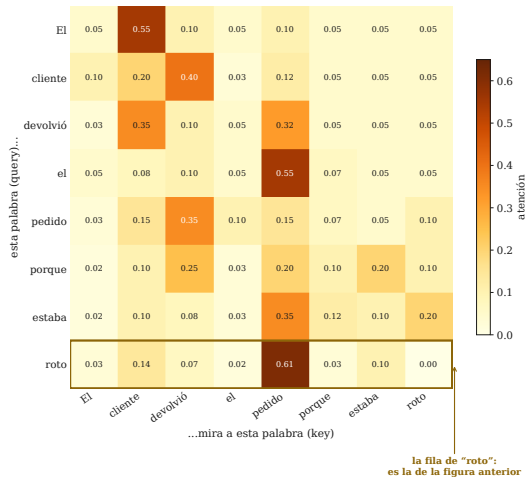
Qué cambia. La palabra entra en el modelo con el mismo vector en los dos casos y sale con vectores distintos, porque la mezcla de valores es distinta.

Enlace con el Día 08. Un embedding fijo da un único vector para "banco" en todo el idioma. La atención da uno por frase.

IDEA CLAVE. Esto es lo que hace un modelo de lenguaje: leer cada palabra a la luz de las que la rodean.

🎯 No solo “roto”: las ocho palabras preguntan a la vez.

Cada palabra de la frase hace su propia pregunta: $8 \times 8 = 64$ pesos.



Cómo se lee. Cada fila es una palabra preguntando; cada columna, una palabra respondiendo. La fila de “roto” es la de las dos figuras anteriores.

Otras filas dicen otras cosas. “devolvió” mira a “cliente” (quién) y a “pedido” (qué). “El” mira a “cliente”, el nombre al que acompaña.

Cada fila suma 1. Ocho palabras, ocho preguntas, $8 \times 8 = 64$ pesos. Con 4 000 palabras serían 16 millones.

IDEA CLAVE. Aquí nace el coste cuadrático: la tabla crece con el cuadrado de la longitud de la frase.

¿Y quién decide lo que busca y lo que ofrece cada palabra? La red.

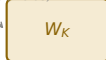
Nadie escribe a mano “pedido = una cosa que se puede romper”: la red lo aprende.

1. el vector de la palabra

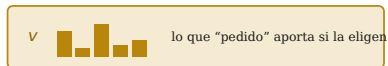
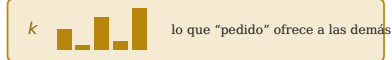
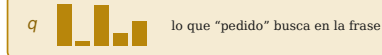


su embedding, el mismo en cualquier frase (Día 08)

2. tres matrices



3. sus tres versiones



Las mismas tres matrices se aplican a todas las palabras: apilando los resultados salen Q , K y V .

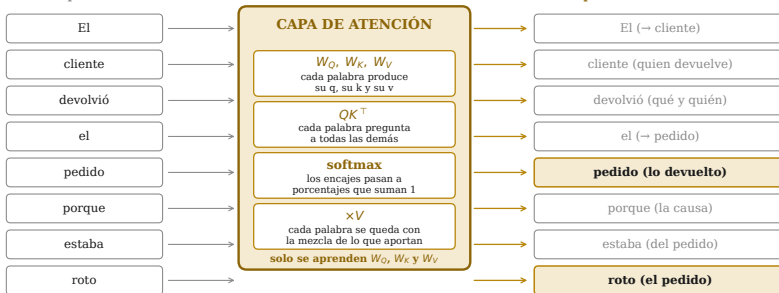
W_Q , W_K y W_V son los pesos de la capa y se aprenden con descenso de gradiente, como los de una capa densa.

IDEA CLAVE. La palabra entra como un vector (su embedding, Día 08) y la capa lo multiplica por tres matrices de pesos. Lo único que se entrena son esas tres matrices; los pesos de atención se recalculan en cada frase. Que “pedido” anuncie “soy algo que se puede romper” es lo que a W_K le salió rentable para acertar.

⚡ La capa de atención, entera: entra una frase y sale la frase con contexto.

La capa de atención: entra un vector por palabra y sale un vector por palabra.

entrada: palabras sueltas



Ocho vectores entran y ocho salen: por eso la atención se apila capa sobre capa (Día 09).

IDEA CLAVE. Es una capa como las densas de los días anteriores: tiene pesos, se entrena con descenso de gradiente y se apila. La diferencia es qué mezcla: una densa mezcla las características dentro de un vector; la atención mezcla los vectores entre sí, y decide la mezcla mirando el contenido de la frase.

Y la fórmula, leída en castellano, no dice nada más que esto.

$$\text{atención de "roto"} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

...y esto es todo lo que dice,
leído de izquierda a derecha:

la pieza

$$QK^T$$



$$\div \sqrt{d_k}$$



softmax



$$\times V$$

qué hace, dicho en castellano

Comparo lo que busca "roto"
con lo que ofrece cada palabra.

Suavizo esa comparación para que
ninguna palabra se lo lleve todo.

Convierto los encajes en porcentajes
de atención que suman 1.

Mezclo lo que aporta cada palabra
según ese porcentaje.

en "El cliente devolvió el pedido porque estaba roto."

"pedido" encaja mucho, "cliente" algo,
"el" y "porque" no encajan.

Sin este paso, "pedido" acapararía
casi el 100 % y el resto se perdería.

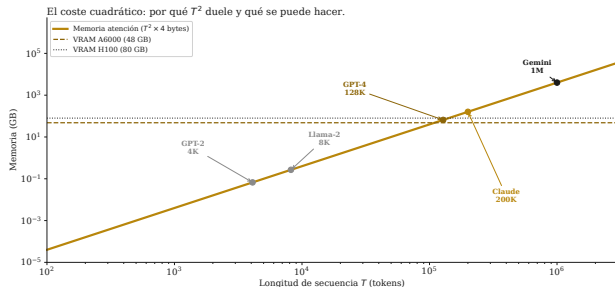
"pedido" 0.61, "cliente" 0.14,
"estaba" 0.10, el resto migajas.

0.61 de "el pedido" + 0.14 de "el cliente"
+ 0.10 de "ocurrió antes" + ...

Resultado: "roto" deja de ser una palabra suelta y pasa a significar "roto, referido al pedido".



El coste cuadrático: por qué T^2 duele y qué se puede hacer.



La ecuación. La matriz de atención QK^T ocupa $T^2 \times 4$ bytes en float32. Para $T = 128\,000$ (GPT-4) eso son ~ 61 GB solo en scores.

Dónde duele. Una A6000 (48 GB) se queda sin memoria a $\sim 110K$ tokens. Para contextos más largos (Claude 200K, Gemini 1M) se necesitan trucos.

Soluciones. KV-cache (guarda keys/values ya calculados), Flash Attention (recomputa en lugar de almacenar), Paged Attention (vLLM), o modelos sub-cuadráticos como Mamba (Día 10).

IDEA CLAVE. El Transformer paga calidad con memoria $O(T^2)$. Los trucos de ingeniería lo hacen viable.

Parte 3 / 4

RNN vs atención, comparativa





Comparativa formal entre familias.

Recurrente (RNN/LSTM/GRU)

Recorre	secuencialmente $t = 1 \dots T$
Memoria	h_t comprime el pasado
Paralelismo	limitado por la cadena temporal
Coste	$\mathcal{O}(T)$ tiempo, $\mathcal{O}(d^2)$ memoria
Brilla en	secuencias cortas, online

Atencional (Transformer)

Recorre	todas las posiciones a la vez
Memoria	matrices Q, K, V explícitas
Paralelismo	total dentro de la secuencia
Coste	$\mathcal{O}(T^2 d)$ tiempo, $\mathcal{O}(T^2)$ memoria
Brilla en	secuencias largas con contexto rico

Concepto → aplicación: cuándo recurrente y cuándo atencional.

Caso	Mejor opción	Por qué	Coste relativo
 Forecast 12 sem. hist. 156 sem.	LSTM/GRU	Memoria suficiente, dataset moderado.	Bajo
 Traducción de un párrafo	 Transformer	Dependencias largas, contexto rico.	Medio
 Asistente convers. 16K tokens	Transformer + KV cache	Posiciones todas a la vez.	Alto
 Audio streaming (latencia ms)	RNN/GRU o SSM (Día 10)	Estado fijo, decisión online.	Bajo
 Informes 500 págs.	Transformer + RAG	Búsqueda + síntesis.	Muy alto

IDEA CLAVE. Recurrente no es obsoleto; sigue ganando en latencia y stream.

Parte 4 / 4

Hoja de ruta



De aquí al final del Bloque I.

De aquí al final del Bloque I: hoja de ruta de los próximos cuatro días.



Lo que el alumno se lleva a casa de este día.

Concepto 1.  Cada palabra se entiende mirando a las de su frase: “roto” reparte su atención y “pedido” se lleva 0.61.

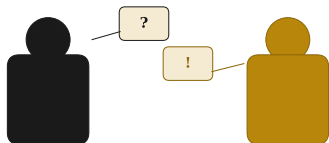
Concepto 2.  Lo que la palabra busca es la query, lo que ofrece cada palabra es el key, lo que aporta es el value. En matrices: $\text{softmax}(QK^T / \sqrt{d_k}) V$.

Concepto 3.  La RNN aprieta toda la historia en un solo h_T ; la atención no comprime nada, pero paga un coste cuadrático (Día 10 ofrece alternativas).

Para la práctica.  Implementar la atención a mano con numpy en 20 líneas y verificarla contra `torch.nn.functional.scaled_dot_product_attention`.

Próximo día.  Día 08 (Tema 3.1): embeddings semánticos.

Pausa para debatir: Si la atención es $O(T^2)$, ¿por qué ganó?



La pregunta. ⚡ El Transformer es caro en memoria. ¿Es una victoria perpetua o un accidente histórico?

Posición A. 👑 Perpetua: ningún reemplazo ha igualado calidad punta.

Posición B. 🚀 Accidente: Mamba y MoE ya lo desplazan en contexto largo (Día 10).

Reglas. 5 minutos, dos turnos de palabra por bando, móviles boca abajo. Yo modero, no participo.

Hasta el Día 08.

Vectores que significan.

Eduardo C. Garrido-Merchán

ecgarrido@comillas.edu

The Google logo is centered within a light beige rounded square. The logo itself consists of the word "Google" in its characteristic multi-colored font: the 'G' is blue, the first 'o' is red, the second 'o' is yellow, the 'g' is blue, the 'l' is green, and the 'e' is red.